

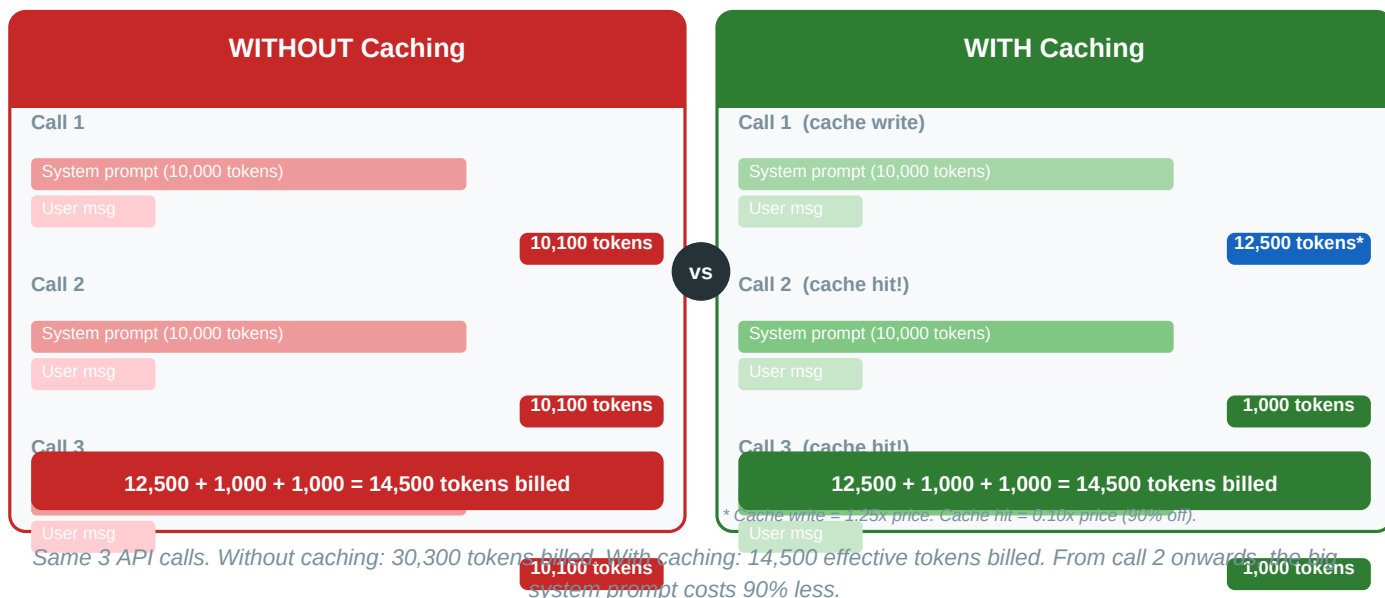
How Prompt Caching Works in Claude

Save up to 90% on repeated large prompts — and get faster responses.

The problem it solves: Every time you call the API, Claude normally re-reads and re-processes your entire prompt from scratch — even if 10,000 tokens of it never change. Prompt caching tells Claude to store that processed context and reuse it on the next call instead of doing the work again. The result: faster responses and a 90% discount on those tokens.

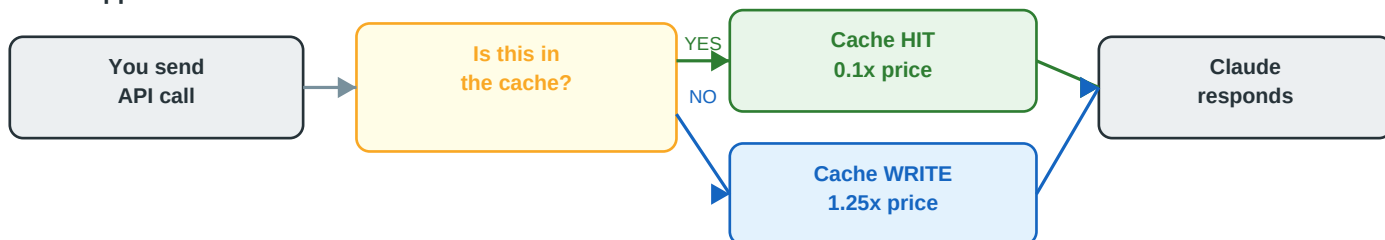
The Core Idea — Same Prompt, Three Calls

Imagine your system prompt is a 10,000-token instruction manual. Without caching, Claude reads the whole manual on every single request. With caching, it reads it once, stores the result, then just glances at the storage on future calls.



What Happens on Each API Call

What happens on each API call



Cache expires after 5 minutes (default) or 1 hour if you request the longer TTL.

Cache hit = Claude skips reprocessing your big context. Faster AND cheaper.

Two Ways to Enable It

Automatic caching (recommended)	Explicit cache breakpoints
Add one cache_control field at the top of your request. Claude automatically figures out which parts to cache as the conversation grows. Best starting point for most apps.	Place cache_control directly on specific content blocks (system prompt, documents, messages). You decide exactly what gets cached. Useful when you need fine-grained control.

Pricing & Cost Savings

Exact numbers from Anthropic's official pricing page, May 2026.

What Each Cache Operation Costs

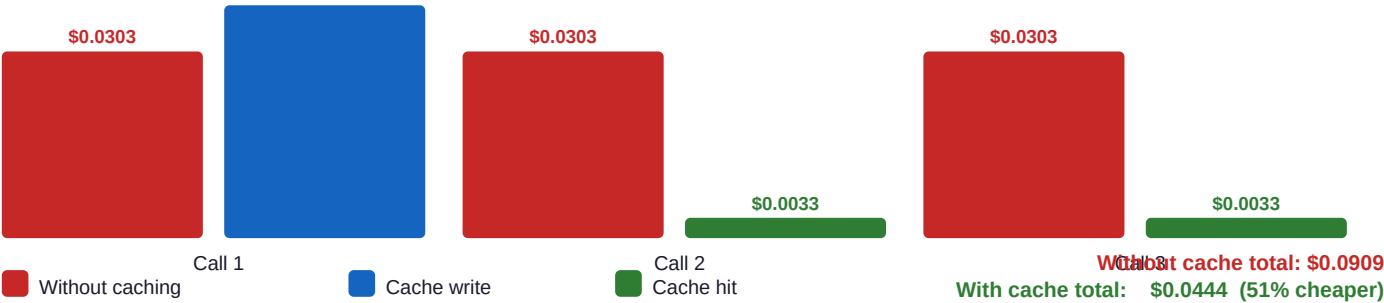
Operation	Cost	Duration	Simple explanation
Normal input (no caching)	1.0x (base price)	—	Claude reads every token fresh on every call.
Cache write (5 minutes)	1.25x base price	Stored for 5 minutes	You pay 25% extra to write to cache. Breaks even after just 1 re-use.
Cache write (1 hour)	2.0x base price	Stored for 1 hour	Costs more to write but stays longer. Breaks even after 2-3 re-uses.
Cache hit (read)	0.1x base price	Same as write TTL	90% cheaper than normal. Claude skips reprocessing entirely.

Break-even rule: The 5-minute cache write costs 1.25x. The cache hit costs 0.1x. So after just **one re-use** within 5 minutes you are already saving money. After 10 re-uses you are paying roughly 20x less than without caching.

Worked Example — 3 Calls with a 10,000-Token System Prompt

Model: Claude Sonnet at \$3.00 / 1M input tokens. System prompt: 10,000 tokens. User message: 100 tokens each call.

Cost comparison — 10,000-token system prompt + 100-token user message, 3 calls



The maths, step by step

Call	What happens	Tokens billed	Cost (no cache)	Cost (with cache)
Call 1	Cache write (5 min). System prompt stored.	$10,000 \times 1.25 + 100 = 12,600$ eff. tokens	\$0.0303	\$0.0378 (write cost)
Call 2	Cache hit. System prompt read from cache at 0.1x.	$10,000 \times 0.10 + 100 = 1,100$ eff. tokens	\$0.0303	\$0.0033 (90% off!)

Call 3	Cache hit again.	Same as Call 2: 1,100 eff. tokens	\$0.0303	\$0.0033 (90% off!)
TOTAL			\$0.0909	\$0.0444 (51% saved)

Code Examples

Two working examples — automatic caching and explicit cache breakpoints.

Example A — Automatic Caching (simplest)

Add one **cache_control** field at the top level. Claude handles the rest. Use this when you have a large system prompt that stays the same across calls.

```
import anthropic

client = anthropic.Anthropic()

BIG_SYSTEM_PROMPT = '...' # your 10,000-token instructions

def ask(user_question: str) -> str:
    response = client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=1024,
        cache_control={'type': 'ephemeral'}, # <-- enables auto caching
        system=BIG_SYSTEM_PROMPT,
        messages=[{'role': 'user', 'content': user_question}]
    )
    return response.content[0].text

# Call 1 – cache WRITE (system prompt stored for 5 min, costs 1.25x)
print(ask('What is the refund policy?'))

# Call 2 – cache HIT (system prompt read from cache, costs 0.1x = 90% off)
print(ask('How do I return a damaged item?'))

# Call 3 – cache HIT again (still within 5 minutes)
print(ask('Can I exchange instead of refund?'))
```

Example B — Explicit Cache Breakpoints

Place **cache_control** on specific content blocks. Here we cache both the system prompt and a large document, then ask different questions about it.

```
import anthropic

client = anthropic.Anthropic()
```

```
LEGAL_DOC = '...' # a 200-page contract (150,000 tokens)
```

```
def ask_about_contract(question: str) -> str:
    response = client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=1024,
        system=[
            {
                "type": "text",
                "text": "You are a legal assistant.",
                "cache_control": {"type": "ephemeral"} # cache system prompt
            }
        ],
        messages=[
            {
                "role": "user",
                "content": [
                    {
                        "type": "text",
                        "text": LEGAL_DOC,
                        "cache_control": {"type": "ephemeral"} # cache document
                    },
                    {
                        "type": "text",
                        "text": question # only this changes each call
                    }
                ]
            }
        ]
    )
    return response.content[0].text

# All three calls cache the 150,000-token document – only charged 0.1x from call 2
print(ask_about_contract('When does this contract expire?'))
print(ask_about_contract('What are the penalty clauses?'))
```

```
print(ask_about_contract('Who are the signatories?'))
```

When Should You Use Prompt Caching?

Use it when...	Example	Why it helps
You have a large system prompt that never changes	A customer support bot with detailed company policies	Every user question reuses the same cached prompt
You are asking multiple questions about the same document	Analysing a contract, codebase, or research paper	Document is cached once, each question is nearly free
You have a long running conversation history	A multi-turn chatbot that has been running for 20+ turns	Old messages stay cached so each new turn is cheap
Do NOT bother if...	Example	Reason
Your prompts are short (under 1,000 tokens)	Simple one-line questions	Cache write overhead is not worth it for small prompts
Each call has a completely different context	Generating unique product descriptions one by one	No repeated content to cache — cache is never hit

Key Takeaways

Claude normally re-reads your full prompt on every API call — even parts that never change

Cache write costs 1.25x the normal price (first call). The cache is stored for 5 minutes by default

Cache hit costs 0.1x the normal price — 90% cheaper. Also returns faster because Claude skips reprocessing

Break-even is after just ONE re-use within 5 minutes. Every call after that is pure savings

Automatic caching (one `cache_control` at the top) is the easiest way to start

Explicit breakpoints give you control over exactly which blocks get cached

Best use cases: large fixed system prompts, repeated document analysis, long chat histories